LABORATORIO DI PROGRAMMAZIONE I

Lezione 3

Liste e File

Laura Nenzi

Many slides by Stefano Russo

Accedere alle stringhe per posizione

Si può accedere all’elemento i-esimo di una stringa così:

```
mia_stringa = "ciao mondo"
mia_stringa[2]   # Terzo carattere della stringa
mia_stringa[-1]  # Ultimo carattere della stringa
```

Lo “slicing” delle stringhe

Si può tagliare una fetta (“to *slice*”) di una stringa così:

```
mia_stringa[0:50]   # Dal primo al cinquantesimo carattere
mia_stringa[30:50]  # Dal trentunesimo al cinquantesimo carattere
mia_stringa[0:-1]   # Dal primo al penultimo carattere
mia_stringa[1:3:2]) # Dal secondo al quarto con intervallo di 2
mia_stringa[-1::-1]) # Dall’ultimo carattere al primo con intervallo di -1
```

(I metodi di una classe)

- Li vedremo più avanti ma intanto un accenno perché potete già usarne
- I metodi di una classe si chiamano mettendoli alla fine della variabile dopo un punto
- Provate ad esempio con un tipo stringa (quindi della classe stringa), scrivendo una stringa ed un punto su VScode compaiono tutti i metodi di quella classe

```
'mia_stringa'.nome_metodo()
```

Liste

- Uno dei tipi dati di Python un po’ più evoluti è la lista.
- Una lista è una sequenza di elementi di qualsiasi tipo

```
my_list = [1,2,3]                    # Lista di numeri
my_list = ['Marco', 'irene', 'paolo']  # Lista di stringhe
my_list = ["ciao", 2.0, 5, [10, 20]] # Lista mista pericolosa!!!
```

Liste: indice

- Come per le stringhe accedo agli elementi con l’indice e posso creare sotto stringhe

```
x = [23,3,2,65, 6,7,8,9,10]
y = x[1:6:2] # dall'elemento di indice 1 all'elemento di indice 6 (escluso), con step 2
z = x[::2]   # dall'inizio alla fine, con step 2.
```

- Con un indice di valore negativo, si conta a ritroso dalla fine della lista.

```
>>> t = ['a', 'b', 'c', 'd', 'e', 'f']
>>> t[-1]
['f']
```

Liste: operazioni e metodi

- Come per le stringhe posso concatenare e ripetere: concatenazione (+), ripetizione (*):

```
>>> print(['a', 'b'] + ['c', 'd', 'e', 'f'])
['a', 'b', 'c', 'd', 'e', 'f']
>>> print(2*['a', 'b'])
['a', 'b', 'a', 'b']
```

- Metodi delle liste, e.g. append:

```
>>> t = ['a', 'b', 'c']
>>> t.append('d')
>>> t
['a', 'b', 'c', 'd']
```

Liste: sono mutabili

- Una lista è mutabile

```
>>> numeri = [42, 123]
>>> numeri[1] = 5
>>> numeri
[42, 5]
```

```
>>> x=[1,2,3]
>>> x[1:3]=[] #elimino gli elementi dall'indice 1 all'indice 3 (escluso)
>>> x
[1]
```

Attenzione!!!

Le stringhe hanno alcune cose in comuni con le liste (slices, molti metodi..)

Ma le **stringhe** sono **immutabili**

```
%%expect TypeError
saluti = 'Hello, world!’
saluti[0] = 'B'
```

Devo creare una nuova stringa

```
nuovi_saluti = 'B' + saluti[1:]
print(nuovi_saluti )
```

Operatori di appartenenza

- Servono a verificare se un elemento è contenuto in una sequenza o in una struttura dati.
- Funzionano con liste, tuple, stringhe, insiemi e dizionari.
- Restituiscono un valore booleano: True oppure False

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

Liste: operatore in e cicli for

```
>>> formaggi = ['Cheddar', 'Edam', 'Gouda']
>>> 'Edam' in formaggi
True
>>> 'Brie' in formaggi
False
```

- Il modo più frequente di attraversare gli elementi di una lista è un ciclo for

```
for formaggio in formaggi:
    print(formaggio)
```

```
for i in range(len(formaggi)):
    print(formaggi[i])
```

```
for i, formaggio in enumerate(formaggi):
    print(i, formaggio)
```

Liste: Attenzione

- A differenza delle stringhe, i metodi delle liste modificano l'oggetto di partenza
- Per puntare ad un oggetto diverso usare .copy() oppure list3=list1[:]

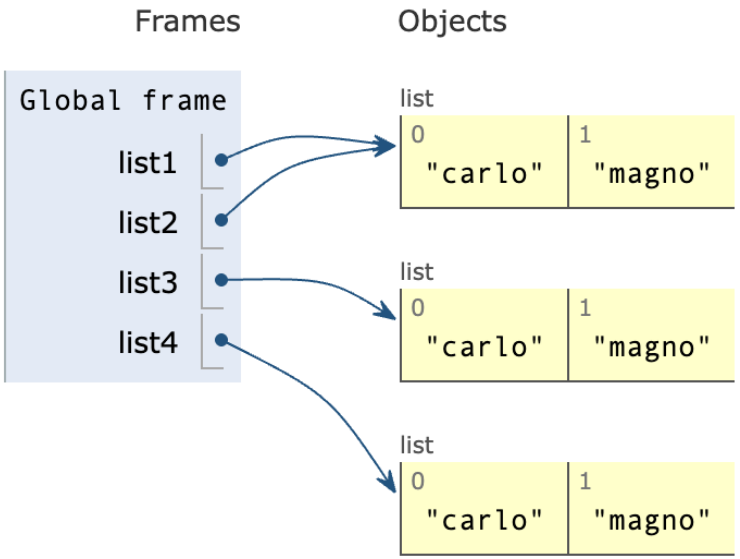
Python 3.11
known limitations

```
1  list1 = ['carlo','magno']
2  list2 = list1
3  list3 = list1[:]
4  list4 = list1.copy()
```

Edit this code

cuted
ute

<< First | < Prev | Next > | Last >>



Liste : Attenzione

- È importante distinguere tra operazioni che modificano le liste e operazioni che creano nuove liste.
- Ad esempio il metodo append non crea una nuova lista

Python 3.11
known limitations

```
1  lista1 = [1, 2, 3]
2  lista1.append(4)
3
4  print(lista1)
5
6  lista2 = [1, 2, 3]
7  nuova_lista = lista2 + [4]
8
9  print(lista2)
10 print(nuova_lista)
```

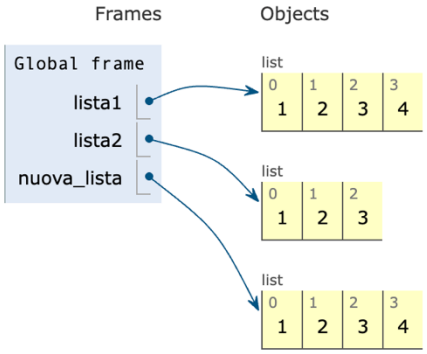
Edit this code

cuted
ite

<< First | < Prev | Next > | Last >>

Print output (drag lower right corner to resize)

```
[1, 2, 3, 4]
[1, 2, 3]
[1, 2, 3, 4]
```



Liste : Attenzione

- Questa differenza è importante quando scrivete delle funzioni che devono modificare delle liste.
- Per esempio, questa funzione non cancella il primo elemento della lista:

```
def non_decapita(t):
    t = t[1:]                # SBAGLIATO!
t4 = [1, 2, 3]
non_decapita(t4)
print(t4)
```

- Un'alternativa valida è scrivere una funzione che crea e restituisce una nuova lista.

```
def nuova_decapitata(t):
    return t[1:]
```

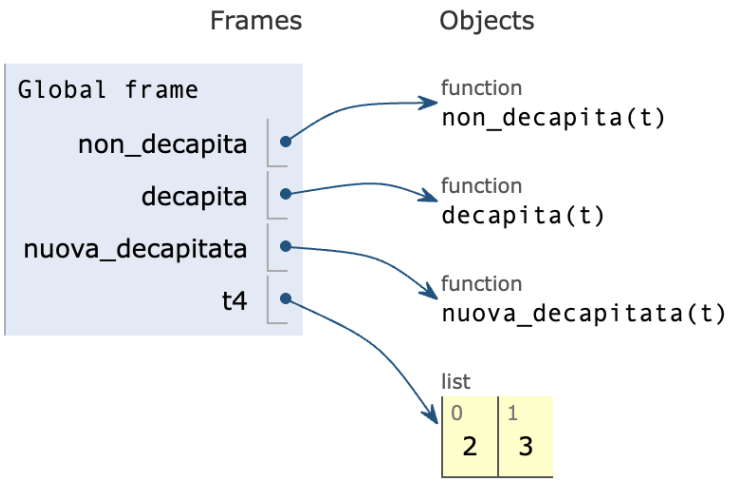
Python 3.11
known limitations

```
def non_decapita(t):
    t = t[1:]                # SBAGLIATO!
def decapita(t):
    t[0:2]=[t[1]]

def nuova_decapitata(t):
    return t[1:]

t4 = [1, 2, 3]

print(non_decapita(t4), t4)
print(decapita(t4),t4 )
print(nuova_decapitata(t4), t4)
```

Edit this code

Print output (drag lower right corner to resize)

```
None [1, 2, 3]
None [2, 3]
[3] [2, 3]
```



La funzione split() in Python

- La funzione split() è un metodo delle stringhe che permette di **dividere una stringa in una lista di sottostringhe**, usando un separatore

```
stringa.split(separatore)
```

- Il separatore è opzionale. Se non specificato, viene usato lo spazio bianco
- Il risultato è sempre una lista di stringhe
- Il separatore viene eliminato

```
data = "2025-03-02"
parti = data.split("-")
print(parti)
```

I dizionari

Un altro tipo dati evoluto di Python sono i “dizionari”. Un dizionario è una **mappa,** coppie chiave-valore. La chiave può essere un qualsiasi oggetto immutabile, come tuple, stringe, interi etc.

```
my_dict = {'Trieste': 34100, 'Padova': 35100}   # stringa -> numero
diz1 = dict()
diz2 = {}
```

Altri esempi di coppie chiave-valore:

```
my_dict = {'Trieste': 34100, 'Padova': 35100}   # stringa -> numero
my_dict = {34100: 'Trieste', 35100: 'Padova'}   # numero -> stringa
my_dict = {'Trieste': 'TS', 'Padova': 'PD'}     # stringa -> stringa
```

I dizionari

Si accede ai valori di un dizionario così:

```
my_dict['Trieste']
```

Posso aggiungere elementi, cambiare valori:

```
my_dict['Venezia'] = 30121                # aggiungere un elemento
my_dict['Venezia']=30100                  # cambiare un valore
```

Se la chiave esiste, il valore viene aggiornato. Se non esiste, viene creata.

Posso aggiungere elementi, cambiare valori e cancellare elementi:

```
del my_dict['Padova']                     # cancellare un elemento
```

I dizionari

Valgono gli operatori di appartenenza della slide precedente, agiscono sulle chiavi

```
if 'Trieste' in my_dict ...
```

Nei dizionari, in verifica la presenza **nelle chiavi**, non nei valori.

```
'Trieste' in my_dict      # True
34100 in my_dict          # False
```

Se voglio verificare la presenza nei valori:

```
34100 in my_dict.values()
```

I dizionari

Posso ottenere la lista di:

- chiave con: `my_dict.keys()`
- valori: `my_dict.values()`
- chiavi-valori: `my_dict.items()`

```
for chiave, valore in my_dict.items():
    print(chiave, valore)
```

- Attenzione questi metodi non restituiscono lista normali, ma oggetti speciale chiamati **view**. Si comportano come una collezione di chiavi, valori, tuple chiavi-valori.
  Se vuoi una lista vera e propria devo fare il cast. E.g.`list(my_dict.keys())`

Le tuple

Liste di valori separati da virgole. Sono immutabili.

```
tupla1 = (2,)
tupla2 = 2,
```

Posso usare le tuple per assegnare più valori contemporaneamente

```
(a,b,c) = (1,2,3)
```

I set

Collezione non ordinate di oggetti unici ed immutabili:

```
a={'a','a','a','c','b','s'} #a=set('s','5')
b = set('abracadabra')
print(type(a),type(b))
```

```
{'a', 'b', 's', 'c'} {'r', 'd', 'c', 'a', 'b'}
```

Ci sono le varie operazioni tra set

```
print ('a - b ', a - b)  # differenza
print ('a | b ', a | b)  # unione, or logico.
print ('a & b ', a & b)  #intersezione, and logico.
print('a^b      ' , a^b)          #simmetric
difference
print('a' in a)
```

Cosa vuol dire essere “pythonici”

Iterare sugli elementi di una lista

```
my_list = ["marco", "irene", "paolo"]

for i in range(len(my_list)):
    print(my_list[i])
```

vs.

```
my_list = ["marco", "irene", "paolo"]

for item in my_list:
    print(item)
```

Cosa vuol dire essere “pythonici”

Controllare se Marco è nella lista degli studenti:

```
my_list = ["marco", "irene", "paolo"]

for i in range(len(my_list)):
    if my_list[i] == "marco":
        print("Ho trovato marco!")
```

vs.

```
my_list = ["marco", "irene", "paolo"]

if "marco" in my_list:
    print("Ho trovato marco!")
```

I moduli

- Un **modulo** è un file `.py` che contiene funzioni, variabili o classi e può essere usato come **libreria** in un altro programma.

- Si importa con la parola chiave **`import`**.

- Potete scrivere anche voi un file .py da importare su un altro script

I moduli

In Python ci sono un sacco di funzionalità (funzioni ed oggetti) già disponibili con la cosiddetta “libreria standard”, ma che non sono “built-in”.

Questo vuol dire che non dovete installare niente per usarle, ma che dovete esplicitamente importare il modulo che le contiene.

Esempio con la radice quadrata:

```
>>> import math
>>> math.sqrt(600)
24.49489742783178
```

oppure

```
>>> from math import sqrt
>>> sqrt(600)
24.49489742783178
```

I file

Per i programmi, uno dei modi più semplici di mantenere i loro dati è di leggerli e scriverli su file di testo.

I files sono molto comodi per salvare dati ancora prima dei database.

Un file di testo è un una sequenza di caratteri salvata su un dispositivo permanente come un disco fisso, una memoria flash.

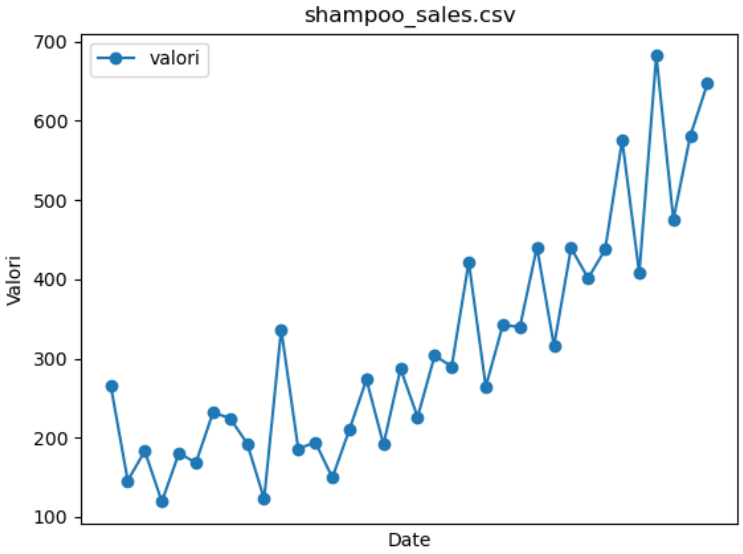
Uno dei formati più standard è il CSV, ovvero “Comma-Separated Values”.

I file

In genere, in un file CSV ogni riga è una “entry”, e ci può essere una intestazione (opzionale) di una o più righe. Estensione .csv o alle volte .txt

Esempio “shampoo_sales.csv”:

```
Date,Sales
01-01-2012,266.0
01-02-2012,145.9
01-03-2012,183.1
01-04-2012,119.3
```



Interagire con i files in Python

Si usa l'oggetto "file". Cosa è un oggetto in dettaglio lo vedremo nella prossima lezione, per ora prendiamolo così com'è.

```
my_file = open('shampoo_sales.csv', 'r')
print(my_file.read())
my_file.close()
```

Modalità di apertura del file, in questo caso "r" sta per "read". Se vorrò anche scriverci, userò "w"

dopo aver lavorato con un file bisogna chiuderlo

```
> python read_file.py
Date,Sales
01-01-2012,266.0
01-02-2012,145.9
01-03-2012,183.1
```

...

Interagire con i files in Python

L’ogetto file ha I seguenti metodi:

```
========= ================================================================
Modalità di accesso
--------- ----------------------------------------------------------------
'r'       lettura (reading) (default)
'w'       scrittura (writing)
'x'       crea un nuovo file e lo apre in scrittura
'a'       scrittura e se il file esiste aggiunge (append) il contenuto in coda al file
'b'       modalità binaria
't'       modalità testuale (default)
'+'       open a disk file for updating (reading and writing)
========= ================================================================
```

Usare lo “slicing” delle stringhe

Si può usare lo slicing delle stringhe per stampare solo un pezzetto del file:

```
my_file = open('shampoo_sales.txt', 'r')
print(my_file.read()[0:50])
my_file.close()
```

```
> python read_file.py
Date,Sales
01-01-2012,266.0
01-02-2012,145.9
01-03
```

Usare lo “slicing” delle stringhe

Si può usare lo slicing delle stringhe per stampare solo un pezzetto del file (versione più sofisticata)

```
# Apro il file
my_file = open('shampoo_sales.txt', 'r')

# Leggo il contenuto
my_file_contents = my_file.read()

# Stampo a schermo i primi 50 caratteri
if len(my_file_contents) > 50:
    print(my_file_contents[0:50] + '...')
else:
    print(my_file_contents)

# Chiudo il file
my_file.close()
```

Leggere i file

Il file si può anche leggere riga per riga, una alla volta:

```
my_file = open('shampoo_sales.csv', 'r')
print(my_file.readline())
print(my_file.readline())
print(my_file.readline())
my_file.close()
```

```
> python read_file.py
Date,Sales

01-01-2012,266.0

01-02-2012,145.9
```

Leggere i file

Il file si può anche leggere riga per riga tutto in un colpo in modo “pythonico”:

```
my_file = open('shampoo_sales.csv', 'r')

for line in my_file:
    print(line)

my_file.close()
```

```
> python read_file.py
Date,Sales

01-01-2012,266.0
```

...

Leggere i file

Per non dover ogni volta chiudere il file posso usare with:

```
with open('shampoo_sales.csv') as file:
        print(file.read())
```

```
with open('shampoo_sales.csv') as file:
        print(file.readline() )
```

Il costrutto **with**

- Il costrutto with funziona con oggetti che implementano il **protocollo di gestione del contesto**, cioè oggetti con i metodi speciali:
  - **__enter__**: eseguito all'inizio del blocco with. Può restituire un oggetto che verrà associato al nome specificato con as.
  - **__exit__**: eseguito alla fine del blocco with, indipendentemente dal fatto che il codice sia terminato correttamente o che abbia sollevato un'eccezione. Qui vengono rilasciate le risorse.
- Usato anche per :

  file compressi (.zip), connessioni a database, connessioni di rete, lock

Cosa succede se non chiudo un file?

Nel momento che chiudo un programma solitamente Python libera tutte le risorse aperte, inclusi i file, e quindi chiude automaticamente i file.

Ma..

- Potrebbero esserci dati non scritti nel buffer che vanno persi
- In Windows può succedere che il file rimanga comunque bloccato

Anche se Python e il sistema operativo gestiscono le risorse alla chiusura del programma, **è una cattiva pratica** lasciare file aperti.

Nomi di file e percorsi

- Il file sono organizzati in directory (chiamate anche “cartelle”).
- Ogni programma in esecuzione ha una “directory corrente”, che è la directory predefinita per la maggior parte delle operazioni che compie.
- Quando aprite un file in lettura, Python lo cerca nella sua directory corrente.
- Il modulo os fornisce delle funzioni per lavorare con file e directory (“os” sta per “sistema operativo”). `os.getcwd` restituisce il nome della directory corrente:

```
>>> import os
>>> cwd = os.getcwd()  #current working directory
>>> cwd
'/Users/laura/git/MyProgrammingLab'
```

Nomi di File e percorsi

- Una stringa come `'/Users/laura/git/MyProgrammingLab'`, che individua la collocazione di un file o una directory, è chiamata percorso.
- Un semplice nome di file, come shampoo_sales.csv è pure considerato un percorso, ma è un percorso relativo perché si riferisce alla directory corrente. Se la directory corrente è '/Users/laura/git/MyProgrammingLab', il nome di file shampoo_sales.csv starebbe per:

  '/Users/laura/git/MyProgrammingLab/shampoo_sales.csv’

- I percorsi visti finora sono semplici nomi di file, quindi sono percorsi relativi alla directory corrente.

Nomi di file e percorsi

- Per avere invece il percorso assoluto, potete usare os.path.abspath:

```
>>> os.path.abspath('shampoo_sales.csv')
'/Users/laura/git/MyProgrammingLab/shampoo_sales.csv'
```

- Se voglio tenere i file dentro una cartella specifica ad esempio "dati" posso inserire il percorso dalla cartella:

```
my_file = open('dati/shampoo_sales.csv', 'r')

for line in my_file:
    print(line)

my_file.close()
```

Scrivere i file

Scrivere su un file (nota il “w” nella funzione open):

```
my_file = open('saluti.txt', 'w')
my_file.write('Ciao mondo!')
my_file.close()
```

.

..ma non lo useremo granchè in questo corso.

Scrivere i file

Scrivere su un file (nota il “w” nella funzione open):

```
my_file = open('saluti.txt', 'w')
my_file.write('Ciao mondo!')
my_file.close()
```

```
with open('saluti.txt', 'w’) as file:
        file.write('Ciao mondo!')
```

..ma non lo useremo granchè in questo corso.

Aggiungere testo su un file

Modificare un file (nota il “a” nella funzione open):

```
my_file = open('saluti.txt', 'a')
my_file.write('Addio!')
my_file.close()
```

```
with open('saluti.txt', 'a' ) as file:
        file.write('Addio!')
```

..ma non lo useremo granchè in questo corso.

Leggiamo i valori di un file CSV

Per leggere i dati da un file CSV bisogna fare un po’ di cose:

1) Il metodo “.split” per separare le stringhe su uno specifico carattere;

2) La conversione di una stringa a valore numerico (floating point);

3) Sapere come aggiungere un elemento ad una lista.

Leggere il valori di un file CSV

1) Il metodo “.split” per separare le stringhe su uno specifico carattere:

```
mia_stringa = 'Date,Sales\n'
lista_elementi = mia_stringa.split(',')
print(lista_elementi)
```

```
> python read_file.py

['Date', 'Sales\n']
```

Leggere il valori di un file CSV

2) La conversione di una stringa a valore numerico (floating point)

```
mia_stringa = '5.5'
mio_numero = float(mia_stringa)
```

Leggere il valori di un file CSV

3) Sapere come aggiungere un elemento ad una lista

```
mia_lista = [1,2,3]
mia_lista.append(4)
```

Leggere il valori di un file CSV

```
# Inizializzo una lista vuota per salvare i valori
values = []

# Apro e leggo il file, linea per linea
my_file = open('shampoo_sales.csv', 'r')
for line in my_file:

   # Faccio lo split di ogni riga sulla virgola
   elements = line.split(',')

   # Se NON sto processando l'intestazione...
   if elements[0] != 'Date':

        # Setto la data e il valore
        date  = elements[0]
        value = elements[1]

        # Aggiungo alla lista dei valori questo valore
        values.append(value)
```

Esercizi

1. Scrivete una funzione che sommi tutti gli elementi di una lista
2. Scrivere una funzione che prende in input una stringa e ritorna True se è un palindromo, False altrimenti.
3. Definire una funzione che prende in input una lista A, indici i, j, e scambia il valore di A[i] con A[j].
4. Scrivere una funzione che prende in input due liste e ritorna True se le due liste hanno almeno un elemento in comune
5. Definire una funzione che prende in input una lista di numeri interi in [0, 9] e ritorna una lista di stringhe, corrispondenti ai numeri scritti in Italiano, es. [1,0,7,9,8] ->["uno","zero","sette","nove","otto"]

Esercizi

1. Scrivere una funzione che prende una lista di parole e restituisce un dizionario con il conteggio delle occorrenze.
2. Definire una funzione che sommi tutti i valori delle vendite degli shampoo del file passato come argomento
3. Definire una funzione che prende in input un file ed una parola e conta quante volte quella parola è presente sul file
4. Definire una funzione conteggio che prende come input un file e ritorna un dizionario con chiave le parole e valore il numero di volte che la parola è presente nel file.
5. Definire una funzione che prende come input un file, rimuove tutte le righe duplicate, scrive il risultato in un nuovo file chiamato unique.txt.